

APAI2025 - LAB04

PULP_NN

Authors: Davide Nadalini, Alberto Dequino, Francesco Conti

Contacts: d.nadalini@unibo.it, alberto.dequino@unibo.it

Links: [GitHub Link \(code\)](#)

Summary

- Subject(s):
 - Parallelization on the PULP architecture
 - Matrix-multiplication
 - 2D conv
 - profiling code execution
- Programming Language: C
- Lab duration: 3h
- 4. Assignment:
 - Time for delivery: 1 week
 - **Submission deadline: Oct 31st 2025**

How to deliver the assignment

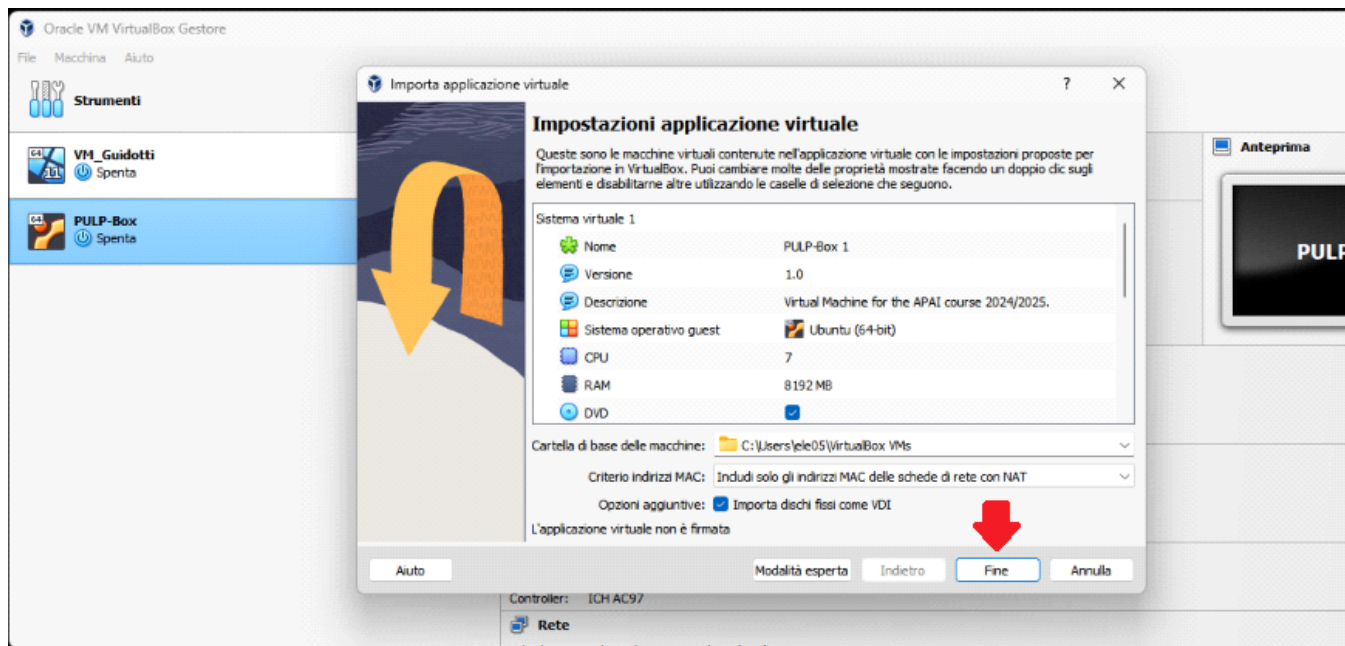
You will deliver ONLY THIS TEXT FILE, no code

- Download this file.
- Fill in the required results.
- Export to pdf format.
- Rename the file to: LAB<number_of_the_lesson>_APAI_<your_name>.pdf
- Use Virtuale platform to load ONLY your .pdf file

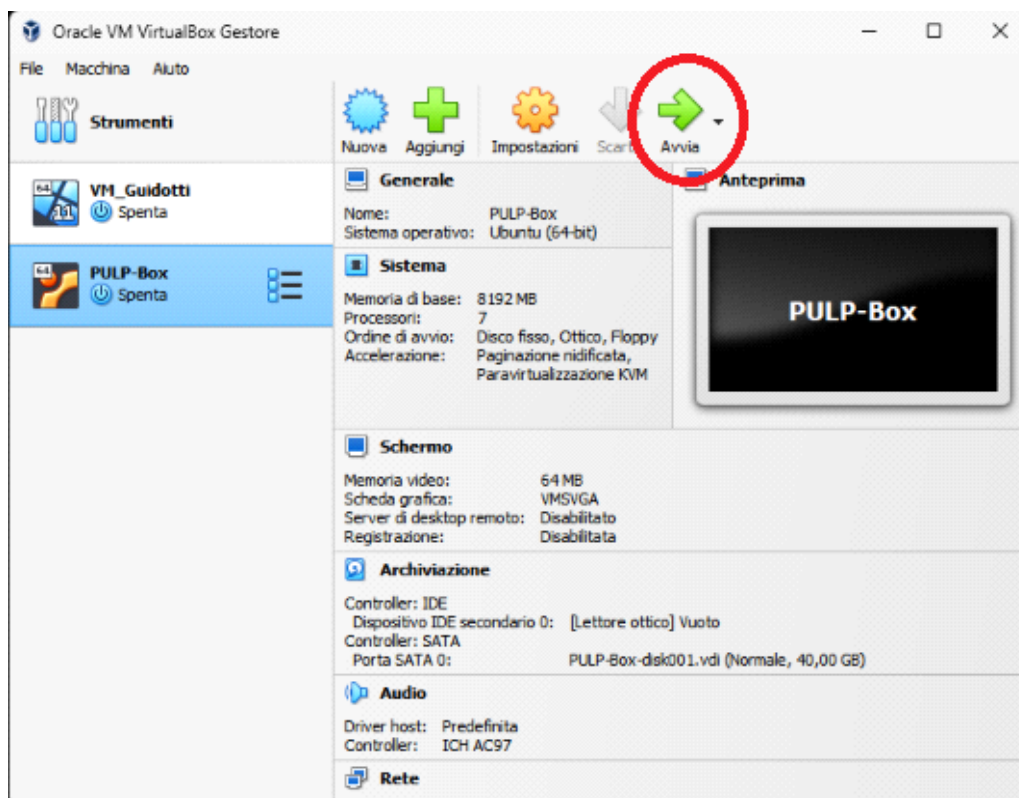
LAB STARTS HERE

0. (ONLY LAB1) Access to the local VM

- On the lab's PCs, open the file explorer and go to This PC, C:/VM_APAI
- Double click on PULP-box.ova
- VirtualBox opens, just click on "Fine"



- Wait for the VM to be imported
- Open the VM with “Avvia”

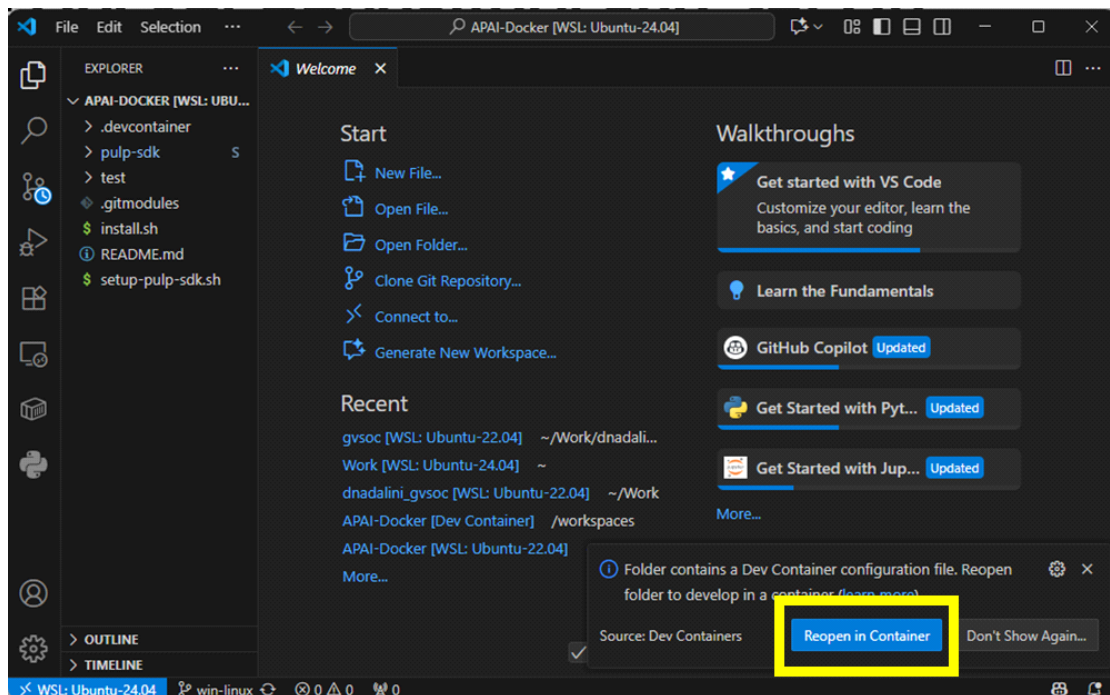


- Password is ‘pulp’

1. (ALL) Open Docker Container with VSCode

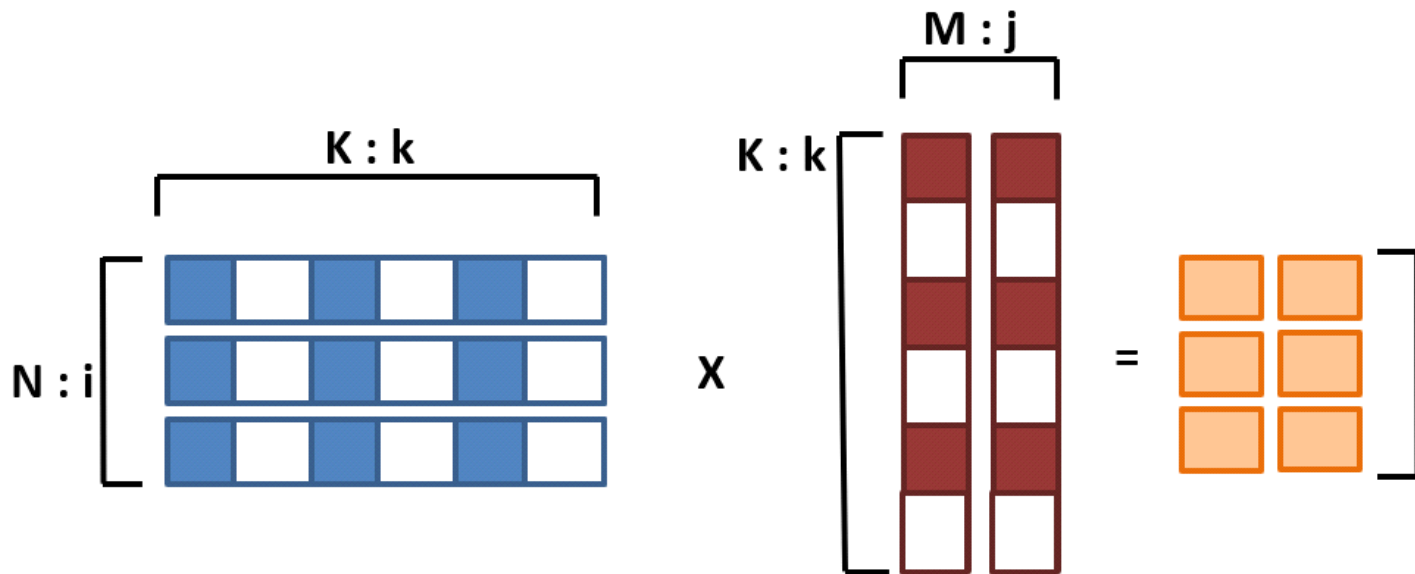
- Open a terminal (right click – open a new terminal)

- Go in the APAI-Docker folder: `cd APAI-Docker`
- Open Docker with VSCode: `code .`
- Select the pop-up option “Reopen in container”



- Setup PULP-SDK: `source setup-pulp-sdk.sh`
- Clone GitHub repository of today's lab: `git clone https://github.com/EEESlab/APAI25-LAB01-PULP-Embedded-Programming`
- `cd APAI25-LAB01-PULP-Embedded-Programming`
- `cd <folder_you_want>`
- `make clean all run`

Task 1: matrix-multiplication



Matrices sizes: $N \times K * K \times M = N \times M$

Initial sizes: $N=32,$ $M=16,$ $K=16$

1.0. Setup:

- Open VSCode (`code .`)
- Go to `matmul_parallelization/` folder
- Every time you want to run the code, **SAVE your file** and write in the terminal :
`make clean all run`
- Don't forget to source the sdk on new terminals:
`source setup_pulp_sdk.sh`

1.1. SPEED-UP and Amdhal's law:

Task Location: `matmul_parallelization/cluster.c`

Setup: $N=32,$ $M=16,$ $K=16$

Sub-tasks:

- Enable performance counters. You will find them in the code → uncomment them
- Fill table below: Calculate execution cycles, and calculate speedup w.r.t. using only one core.

Tip: for executing with different numbers of cores, use the “CORES” flag.

Example: `make clean all run CORES=80.66`

CORES	Cycles	Speedup (w.r.t. CORES=1)
1	39568	1
2	19825	1.98
4	10484	3.76
8	6108	6.60

1.2. Explore different input sizes:

Task Location: `matmul_parallelization/cluster.c`

Setup:

- 8 CORES
- N=<varying> M=16 K=16

Sub-tasks:

- measure performance of **each individual core** (Execution cycles, and IPC)

Matrix size: N	Instructions executed (each individual core)							
	0	1	2	3	4	5	6	7
4	1017	1017	1017	1017	35	35	35	35
8	1017	1017	1017	1017	1017	1017	1017	1017
80	9766	9766	9766	9766	9766	9766	9766	9766
81	10738	10738	10738	10738	10738	10738	10738	3934

Matrix size: N	IPC (each individual core)							
	0	1	2	3	4	5	6	7
4	0.73	0.72	0.73	0.73	0.02	0.03	0.03	0.03
8	0.52	0.52	0.52	0.52	0.52	0.52	0.52	0.52
80	0.69	0.69	0.69	0.69	0.69	0.69	0.69	0.69
81	0.71	0.71	0.71	0.71	0.71	0.71	0.71	0.26

Answer the following questions:

- Is the workload equally balanced with N=4? Why ?
- Is the workload equally balanced with N=8? Why ?
- Is the IPC higher for N=80 or N=81? Why ?

[ANSWER]:

- No, the workload is not balanced for N=4. Only cores 0–3 execute a substantial number of instructions, while cores 4–7 perform almost none. The issue arises because the number of rows (4) is lower than the number of available cores (8).
- Yes, the workload is not balanced for N=8. This happens as the number of rows is exactly as the number of cores.
- The IPC for N=81 is marginally higher than for N=80, but this does not reflect improved efficiency. At N=81, core 7 displays a noticeably lower IPC value (0.26), indicating uneven load distribution. Because 81 rows cannot be evenly divided across 8 cores, the extra row causes one core to complete its work at a different time, exposing imbalance and reducing efficiency on that core.

- measure **the overall** performance of the GEMM: Cycles, MACs, MAC/Cycles

Tip: Calculate the MACs by hand ($N \cdot M \cdot K$)

N	MACs	Cycles (equal to all cores)	MAC/cycles (equal to all cores)
4	1024	1394	0.735
8	2048	1948	1.051
80	20480	14111	1.451
81	20736	15026	1.380

Answer the following questions:

- Why when N=81 the MAC/cycles is lower than when N=80?

[ANSWER]:

- The extra row introduces imbalance and overhead, increasing total cycles more than the increase in MACs. The result is lower MAC per cycle despite nearly identical computational work.

1.3. Load stalls & unrolling the MatMul

Task Location:

- `matmul_parallelization/cluster.c`
- `matmul_parallelization/matmuls.c`

Setup:

- 8 CORES
- N=32 M=16 K=16

Sub-task:

- Enable the performance counters of our interest. We want to profile:
 - Execution cycles (total)
 - N° instructions executed
 - Load Stalls → **missing**

Complete the code where you find `/* YOUR CODE HERE */` with the right performance counters

```
pi_perf_conf(
    1 << PI_PERF_CYCLES
    | 1 << PI_PERF_INSTR
    // /*YOUR CODE HERE*/ //(add load stall)
);
```

Put your solution below (code)

```
pi_perf_conf(
    1 << PI_PERF_CYCLES |
    1 << PI_PERF_INSTR |
    1 << PI_PERF_LD_STALL
);
```

Tip: Here's the full list of the performance counters

```

typedef enum {
    PI_PERF_CYCLES      = 17, /*!< Total number of cycles (also includes the
        cycles where the core is sleeping). Be careful that this event is using a
        timer shared within the cluster, so resetting, starting or stopping it on
        one core will impact other cores of the same cluster. */
    PI_PERF_ACTIVE_CYCLES = 0, /*!< Counts the number of cycles the core was
        active (not sleeping). */
    PI_PERF_INSTR        = 1, /*!< Counts the number of instructions executed.
        */
    PI_PERF_LD_STALL     = 2, /*!< Number of load data hazards. */
    PI_PERF_JR_STALL     = 3, /*!< Number of jump register data hazards. */
    PI_PERF_IMISS        = 4, /*!< Cycles waiting for instruction fetches, i.e.
        number of instructions wasted due to non-ideal caching. */
    PI_PERF_LD           = 5, /*!< Number of data memory loads executed.
        Misaligned accesses are counted twice. */
    PI_PERF_ST           = 6, /*!< Number of data memory stores executed.
        Misaligned accesses are counted twice. */
    PI_PERF_JUMP         = 7, /*!< Number of unconditional jumps (j, jal, jr,
        jalr). */
    PI_PERF_BRANCH       = 8, /*!< Number of branches. Counts both taken and
        not taken branches. */
    PI_PERF_BTAKEN       = 9, /*!< Number of taken branches. */
    PI_PERF_RVC          = 10, /*!< Number of compressed instructions
        executed. */
    PI_PERF_LD_EXT       = 12, /*!< Number of memory loads to EXT executed.
        Misaligned accesses are counted twice. Every non-TCDM access is considered
        external (cluster only). */
    PI_PERF_ST_EXT       = 13, /*!< Number of memory stores to EXT executed.
        Misaligned accesses are counted twice. Every non-TCDM access is considered
        external (cluster only). */
    PI_PERF_LD_EXT_CYC   = 14, /*!< Cycles used for memory loads to EXT.
        Every non-TCDM access is considered external (cluster only). */
    PI_PERF_ST_EXT_CYC   = 15, /*!< Cycles used for memory stores to EXT.
        Every non-TCDM access is considered external (cluster only). */
    PI_PERF_TCDM_CONT    = 16, /*!< Cycles wasted due to TCDM/log-interconnect
        contention (cluster only). */
} pi_perf_event_e;

```

Ref: `/rtos/pmsis/pmsis_api/include/pmsis/chips/default.h`

- Implement missing code on the `gemm_unroll()`. Then manually fix the code in order to unroll 2-4-8-16 operations.
- Fill in now the table, comparing the naive `gemm()` vs. the unrolled version `gemm_unroll()`

	stalls	Instructions	Cycles	IPC	MACs	MAC/Cycles
naive	1024	3935	6041	0.65	8192	1.356
Unrolled 2	0	4204	5807	0.72	8192	1.411
Unrolled 4	0	4543	5764	0.79	8192	1.421
Unrolled 8	0	5423	6676	0.81	8192	1.227
Unrolled 16	896	10960	13699	0.80	8192	0.598

Answer the following questions:

- Compare the stalls of “naive” vs. unrolled (2-4-8). Who has more? Why?

- Why Unrolled16 has more stalls than Unrolled8?

[ANSWER]:

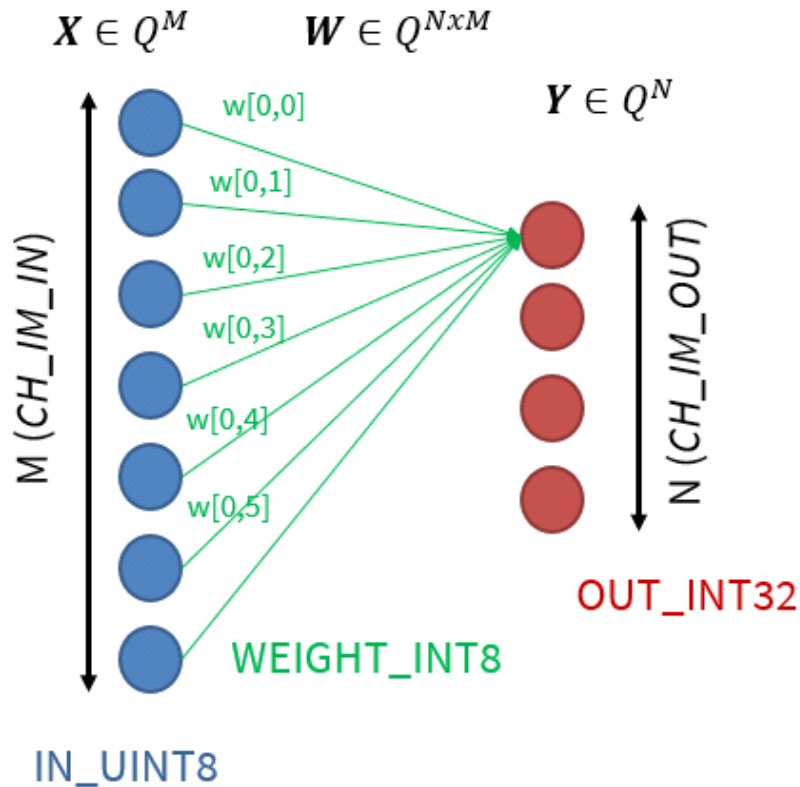
- The naive loop incurs more stalls than the unrolled implementations. Each iteration performs a load directly before the MAC, so the processor must wait for the load to complete. This introduces a one-cycle dependency stall on every iteration.
- The Unrolled16 implementation incurs more stalls than Unrolled8 because it surpasses the optimal level of unrolling. The loop body becomes so large that it requires more intermediate values than the core's register file can efficiently hold, creating register pressure. Concurrently, the increased number of memory operations in a single loop iteration can overwhelm the memory subsystem's bandwidth. So the processor stalls not from the latency of individual operations, but from structural hazards caused by the inflated unrolled body.

1.4. (optional) Data Reuse

Read instructions inside the code

FC	IPC	MACs/cycle
naive	0.66	2.08
Reuse 2	0.77	1.96
Reuse 4	0.92	2.52
Reuse 8	0.96	2.92

Task 2: Fully Connected Layer



2.0. Setup:

- Open VSCode.
- Go to `fully_connected/` folder
- Every time you want to run the code, **SAVE your file** and write in the terminal :
`make clean all run`
- Don't forget to source the sdk on new terminals:
`source setup_pulp_sdk.sh`

2.1. MACs: FullyConnected without vs. with SIMD

Compare the number of executed instructions in the normal FullyConnected layer and the SIMD version.

- Calculate the number of MACs by hand
- Use perf counter to measure the number of executed instructions
- Change the `dotp_u8_i8_i32` function to the `dotp_u8_i8_i32_simd` and measure the number of executed instructions again

Note: the number of executed MAC operations is dictated by the size of the FullyConnected layer, not the way operations are implemented (non-SIMD/SIMD). The formula to calculate the total number of MAC operations is:

$$\text{MAC} = C_{\text{in}} \times C_{\text{out}}$$

You can find the FullyConnected layer dimensions in the *data_allocation.h* header file.

TIP: 8-bit SIMD instructions perform 4 MACs in 1 cycle

Cores (#N)	SIMD	Operations (MAC)	Instructions (#N)
1	No	16384	49362
1	Yes	16384	12534

What is the ratio of instructions before/after the new SIMD implementation? Why?

[ANSWER]:

- The instruction count ratio is 3.94 (≈ 4). This improvement stems from the data parallelism of the SIMD implementation. The processor uses 32-bit registers to operate on four 8-bit data elements simultaneously within a single instruction. Therefore, for the main computational kernel, the new SIMD version requires only one instruction to accomplish the work of four separate instructions in the original, scalar code.

2.2. Calculate speedup

Measure the latency of a FullyConnected layer and fill out the table below.

From the measured latency calculate the performance and speedup with regard to the single core latency.

Note: to calculate the performance you will have to divide the total number of MAC operations with the measured latency.

Cores (#N)	SIMD	Latency (cycles)	Performance (MAC/cycle)	Speedup w.r.t #Cores=1 SIMD=No
1	No	65916	0.25	1.0x
1	Yes	16813	0.97	3.9x
8	No	8402	1.95	7.85x
8	Yes	2210	7.41	29.83x

Task 3: Convolution Layer

3.0. Setup

- Open VSCode.
- Go to **convolution/** folder
- Every time you want to run the code, **SAVE your file** and write in the terminal :
`make clean all run`
- Don't forget to source the sdk on new terminals:
`source setup_pulp_sdk.sh`

3.1. Speedup over multiple cores

Measure the latency of a Convolution layer and fill out the table below. From the measured latency calculate the performance and speedup with regard to the single core latency.

Note: to calculate the performance you will have to divide the total number of MAC operations with the measured latency. The formula to calculate the total number of MAC operations is:

$$\text{MACs} = \text{DIM_IM_OUT_X} * \text{DIM_IM_OUT_Y} * \text{CH_IM_OUT} * \text{CH_IM_IN} * \text{DIM_KERNEL_X} * \text{DIM_KERNEL_Y}$$

You can find the Convolution layer dimensions in the *data_allocation.h* header file.

Cores (#N)	MACs	Latency (cycles)	Performance (MAC/cycle)	Speedup w.r.t #Cores=1 SIMD=No
1	4.718.592	2335845	2.02	1.0x
2	4.718.592	1170068	4.03	2.0x
4	4.718.592	588612	8.02	3.97x
8	4.718.592	298695	15.80	7.82x